

Project 8: 摄像机标定实验报告

姓名: 李祥熙 学号: 2234412799 班级: 计算机 2301

目录

1	实验目的	2
2	实验环境	2
3	实验过程	2
4	摄像机标定理论	2
4.1	径向畸变	3
4.2	切向畸变	3
5	相机标定结果	3
5.1	角点检测	3
5.2	内参矩阵	4
5.3	畸变系数	4
5.4	外参 (第一张图像)	4
6	畸变矫正结果	4
7	参考文献	5
A	附录: 标定代码	6
A.1	生成棋盘格图像	6
A.2	角点检测与摄像机标定	6
A.3	畸变矫正	7

1 实验目的

- **理解摄像机标定原理**：掌握张正友标定法（Zhang's method）的基本理论，了解内参矩阵、畸变系数与外参的含义。
- **熟悉 OpenCV 标定流程**：使用 OpenCV 提供的 `calibrateCamera` 函数，对真实手机镜头进行标定。
- **分析镜头畸变特性**：通过对标定结果的分析，理解径向畸变与切向畸变在实际镜头中的表现。
- **实现畸变矫正**：利用标定得到的内参和畸变系数，对真实图像进行畸变矫正，验证标定结果的有效性。

2 实验环境

- 操作系统：Linux（Arch Linux）
- 编程语言：Python 3
- 主要工具：Jupyter Notebook、OpenCV、NumPy
- 拍摄设备：

3 实验过程

基本实验过程如下：

1. 利用 Python 脚本（`checkerboard.py`）生成棋盘格图像并打印；
2. 拍摄若干张不同角度和距离的棋盘格图像，关闭手机自带的镜头矫正功能；
3. 运行 `calibration.ipynb` 进行角点检测和摄像机标定，获得内参、畸变系数和外参；
4. 利用内参和畸变系数对另一张图像进行畸变矫正。

4 摄像机标定理论

本实验使用 OpenCV 实现的张正友标定法。OpenCV 中的畸变模型包含径向畸变和切向畸变两部分。

4.1 径向畸变

径向畸变由镜头形状引起，其模型为

$$x_{\text{distorted}} = x(1 + k_1r^2 + k_2r^4 + k_3r^6), \quad y_{\text{distorted}} = y(1 + k_1r^2 + k_2r^4 + k_3r^6) \quad (1)$$

其中 $r^2 = x^2 + y^2$, (x, y) 为归一化图像坐标, k_1, k_2, k_3 为径向畸变系数。

4.2 切向畸变

切向畸变由镜头与成像平面不完全平行引起，其模型为

$$x_{\text{distorted}} = x + [2p_1xy + p_2(r^2 + 2x^2)], \quad y_{\text{distorted}} = y + [p_1(r^2 + 2y^2) + 2p_2xy] \quad (2)$$

其中 p_1, p_2 为切向畸变系数。综合两种畸变，最终的畸变系数为五元组

$$\text{Distortion coefficients} = (k_1 \quad k_2 \quad p_1 \quad p_2 \quad k_3) \quad (3)$$

5 相机标定结果

5.1 角点检测

对每张棋盘格图像进行 9×6 内角点检测，结果如图 1 所示。

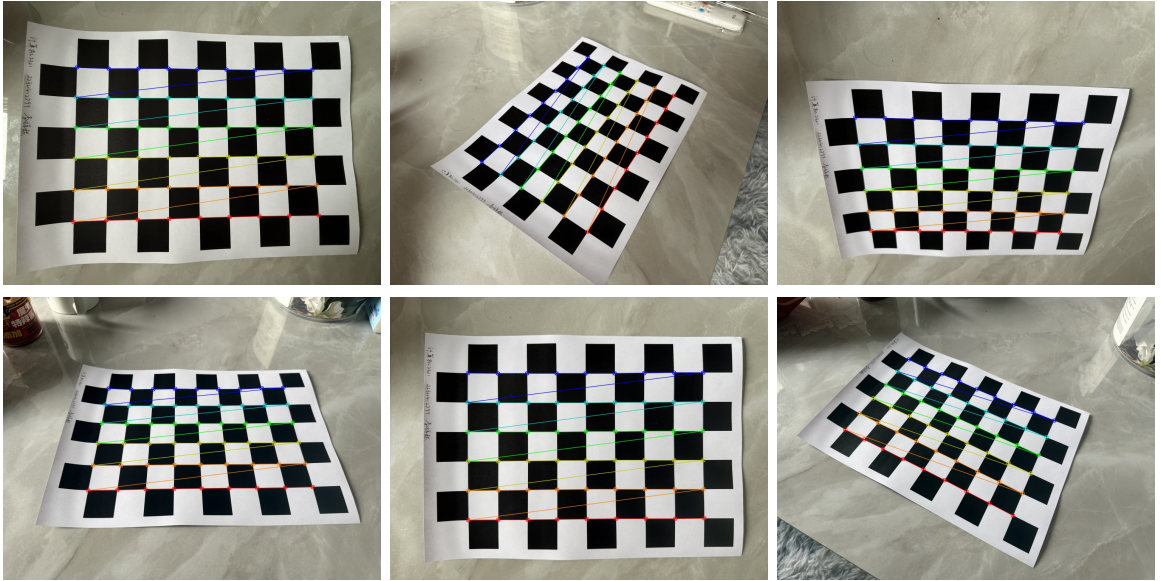


图 1: 各标定图像的角点检测结果

5.2 内参矩阵

标定得到的相机内参矩阵 K 如下：

$$K = \begin{pmatrix} 733.39419537 & 0 & 503.82907973 \\ 0 & 732.11725156 & 369.89539954 \\ 0 & 0 & 1 \end{pmatrix} \quad (4)$$

5.3 畸变系数

表 1: 畸变系数

k_1	k_2	p_1	p_2	k_3
0.24604382	-1.01102813	-0.00450469	0.00264820	1.64885357

5.4 外参（第一张图像）

对应第一张标定图像的旋转向量 r_1 和平移向量 t_1 如下：

$$r_1 = \begin{pmatrix} -0.05866420 \\ 0.94653745 \\ 2.97498748 \end{pmatrix}, \quad t_1 = \begin{pmatrix} 4.28616412 \\ 1.85310577 \\ 10.23931967 \end{pmatrix} \quad (5)$$

其中旋转向量为 Rodrigues 形式，可通过 `cv2.Rodrigues()` 转换为旋转矩阵。

6 畸变矫正结果

使用一张另拍摄的未矫正图像（图 2），利用标定得到的内参和畸变系数进行畸变矫正。



图 2: 原始未矫正图像

畸变矫正后（保留完整画面）的结果如图 3 所示，对无效区域裁剪后的结果如图 4 所示。



图 3: 畸变矫正后（含黑色边缘区域）



图 4: 畸变矫正后（裁剪至有效区域）

7 参考文献

1. OpenCV Camera Calibration Tutorial, https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html
2. OpenCV Rodrigues, https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html#ga61585db663d9da06b68e70cfbf6a1eac

A 附录：标定代码

A.1 生成棋盘格图像

```
from PIL import Image

h, hc = 2000, 10
w, wc = 1400, 7
img = Image.new("RGB", (w, h), (0, 0, 0))
pixels = img.load()
for i in range(w):
    for j in range(h):
        if ((i // (h // hc)) & 1) ^ ((j // (w // wc)) & 1):
            pixels[i, j] = (0, 0, 0)
        else:
            pixels[i, j] = (255, 255, 255)
img.save('./checkerboard.png')
```

A.2 角点检测与摄像机标定

```
import cv2, numpy as np, glob

CHECKERBOARD = (9, 6)
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 50,
            0.001)
objectp3d = np.zeros((1, CHECKERBOARD[0] * CHECKERBOARD[1], 3), np.
                    float32)
objectp3d[0, :, :2] = np.mgrid[0:CHECKERBOARD[0], 0:CHECKERBOARD[1]].T.
                    reshape(-1, 2)

threedpoints, twodpoints = [], []
for i, filename in enumerate(glob.glob('./checkerboards/*.jpg')):
    image = cv2.resize(cv2.imread(filename), (1000, 750))
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    ret, corners = cv2.findChessboardCorners(gray, CHECKERBOARD,
        cv2.CALIB_CB_ADAPTIVE_THRESH + cv2.CALIB_CB_FAST_CHECK +
        cv2.CALIB_CB_NORMALIZE_IMAGE)
    if ret:
        threedpoints.append(objectp3d)
        corners2 = cv2.cornerSubPix(gray, corners, (20, 20), (-1, -1),
```

```
        criteria)
    twodpoints.append(corners2)
    cv2.drawChessboardCorners(image, CHECKERBOARD, corners2, ret)
    cv2.imwrite(f'./corner/{i}.jpg', image)

ret, matrix, distortion, r_vecs, t_vecs = cv2.calibrateCamera(
    threedpoints, twodpoints, gray.shape[::-1], None, None)
print("Camera matrix:\n", matrix)
print("Distortion coefficients:\n", distortion)
```

A.3 畸变矫正

```
import cv2, glob

for i, filename in enumerate(glob.glob('./samples/*.jpg')):
    img = cv2.resize(cv2.imread(filename), (1000, 750))
    h, w = img.shape[:2]
    newmtx, roi = cv2.getOptimalNewCameraMatrix(matrix, distortion, (w,
        h), 1, (w, h))
    dst = cv2.undistort(img, matrix, distortion, None, newmtx)
    cv2.imwrite(f'./undistorted/calibresult_nocrop_{i}.png', dst)
    x, y, w, h = roi
    cv2.imwrite(f'./undistorted/calibresult_{i}.png', dst[y:y+h, x:x+w
        ])
```